MARKETPULSE:

AUTOMATED S&P 500 MARKET INTELLIGENCE DASHBOARD

Built with Python · Power BI Desktop · GitHub Actions

Prepared by: Naveen Karan Krishna

Date: April 2026

Version: 1.0

Contact: naveenxkaran@gmail.com |

https://www.linkedin.com/in/naveen-karan-krishna/

Portfolio: https://github.com/NK-Mikey

TABLE OF CONTENTS

1. EXECUTIVE SUMMARY

MarketPulse is an automated stock market intelligence dashboard that tracks the Top 25 S&P 500 companies by market capitalization, delivering daily updated financial analytics through a professionally designed Power BI interface backed by a fully automated Python data pipeline.

The project was designed to replicate the core functionality of platforms like Yahoo Finance and Bloomberg Terminal using exclusively free tools, specifically Python, Power BI Desktop, and GitHub Actions. This demonstrates that enterprise-grade analytics workflows can be built and maintained without paid data subscriptions or licensed platforms.

Key Deliverables:

Deliverable	Description
Automated ETL Pipeline	Daily data extraction from Yahoo Finance via Python and GitHub Actions
Power BI Dashboard	Interactive financial dashboard with 6 distinct analytical views
Data Model	Snowflake schema with 4 core tables and 2 disconnected parameter tables
Documentation	Full technical documentation, data dictionary, runbook and SOP
GitHub Repository	Version-controlled codebase with automated daily refresh

Outcome:

A portfolio-grade, end-to-end data analytics solution that demonstrates proficiency across data engineering, data modeling, DAX development, dashboard design, and pipeline automation, all built within a constrained free toolset.

2. PROBLEM STATEMENT

Background:

Financial market data is widely available but operationally inaccessible for most analysts. Platforms like Bloomberg Terminal cost upwards of $25,000 per year. Yahoo Finance provides the data but no analytical interface. Most data analysts working in non-finance domains lack a practical way to demonstrate financial analytics skills without access to expensive tools or proprietary datasets.

The Challenge:

Three specific problems motivated this project:

Problem 1: Data accessibility without cost Real-time and daily stock market data exists freely via Yahoo Finance but requires engineering effort to extract, structure and refresh automatically. Without automation, daily manual data collection is not sustainable.

Problem 2: Portfolio gap in financial analytics Junior data analyst portfolios typically demonstrate either Python skills or BI skills in isolation. Few demonstrate an integrated end-to-end workflow that combines automated data engineering with professional dashboard design, which is the combination most employers actually require.

Problem 3: Tool constraints Power BI Pro licensing ($10 per user per month) gates many advanced features including scheduled cloud refresh and dashboard sharing. The challenge was to build a production-quality workflow within the free tier, finding professional workarounds where licensing limitations existed.

Objective:

Build a fully automated, professionally documented stock market analytics dashboard using only free tools that demonstrates end-to-end data analyst competencies, from raw API data to executive-ready visualization, and can be maintained and handed over to another analyst without additional explanation.

3. SOLUTION OVERVIEW

Approach:

MarketPulse was built as a three-layer solution:

Layer 1: Data Layer (Python) Automated extraction of daily stock prices, company fundamentals, and S&P 500 metadata from Yahoo Finance and Wikipedia. Outputs three structured CSV files on a daily schedule.

Layer 2: Automation Layer (GitHub Actions) A scheduled workflow runs every day at 6:00 AM UTC, executing the Python extraction script and committing updated data back to the GitHub repository automatically.

Layer 3: Analytics Layer (Power BI) A professionally designed dashboard reads the latest data directly from GitHub raw URLs, requiring only a single manual refresh keystroke (Alt + F5) to display fully updated market intelligence.

Why This Approach:

Decision	Rationale
Python over manual downloads	Eliminates daily manual work and demonstrates automation skills
GitHub Actions over local scheduler	Free, cloud-based, version-controlled and portfolio-visible
GitHub CSV storage over local files	Any analyst with Power BI Desktop can refresh without local data
Snowflake schema over flat table	Demonstrates professional data modeling knowledge
Top 25 by market cap over hardcoded list	Dynamic, self-updating and more realistic than arbitrary selection
Power BI Desktop over paid BI tools	Demonstrates ability to deliver results within real constraints

4. DATA ARCHITECTURE

Data Model Overview:

The project uses a **snowflake schema**, which is an extension of the star schema where dimension tables are further normalized. This was chosen over a flat table or pure star schema to cleanly separate price data, company metadata, and financial fundamentals into distinct domains.

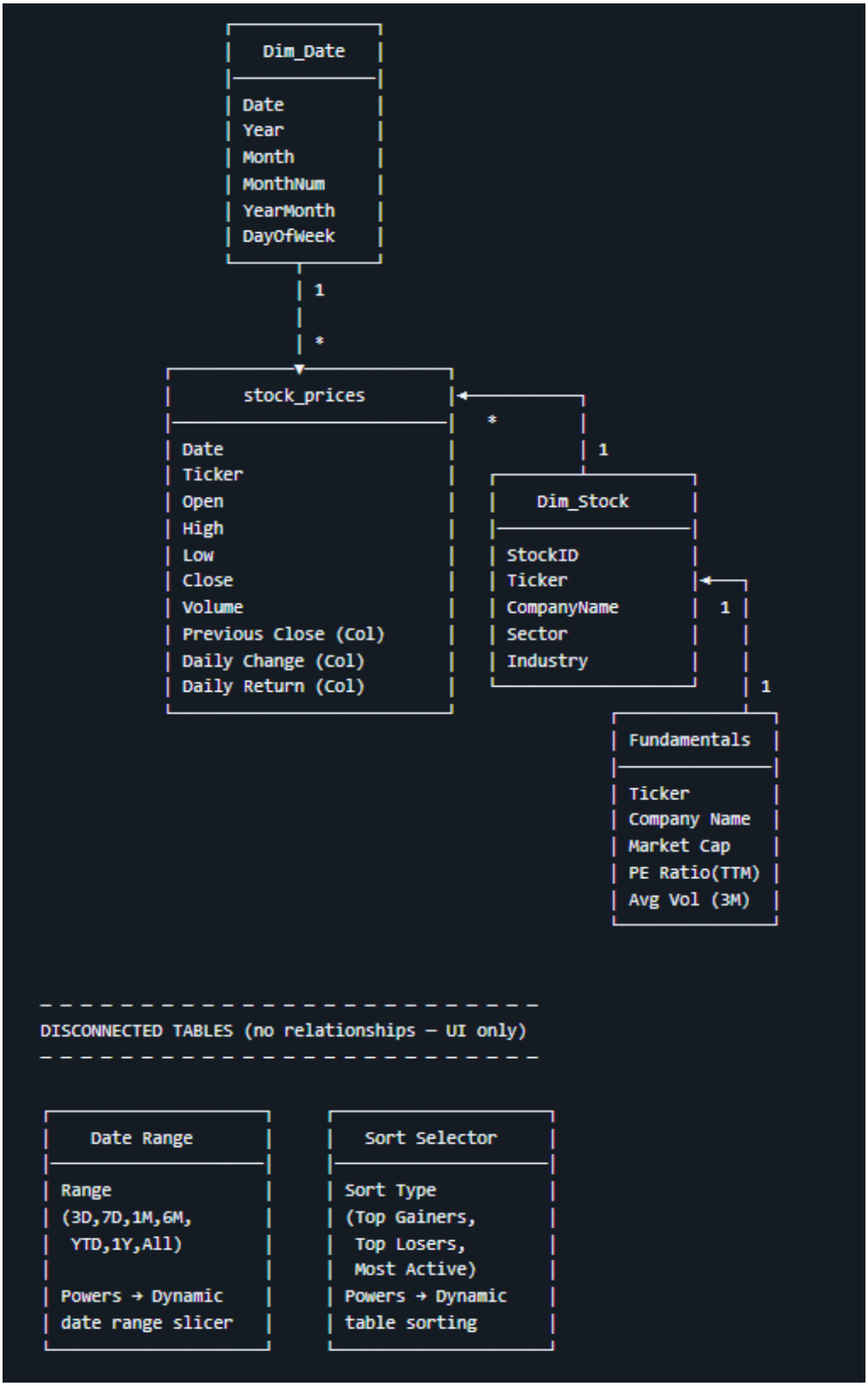
Table Descriptions:

Table	Type	Rows	Purpose
stock_prices	Fact	~18,500	Daily OHLCV price data for all 25 tickers from Jan 2023
Dim_Stock	Dimension	25	Company name, sector, industry lookup
Fundamentals	Dimension	25	Market cap, PE ratio, average volume
Dim_Date	Date Dimension	~1,100	Calendar table for Power BI time intelligence
Date Range	Disconnected	7	Drives dynamic date range slicer
Sort Selector	Disconnected	3	Drives Top Gainers / Losers / Most Active sorting

Relationships:

From	To	Cardinality	Direction
Dim_Date[Date]	stock_prices[Date]	One to many	Single
Dim_Stock[Ticker]	stock_prices[Ticker]	One to many	Single
Fundamentals[Ticker]	Dim_Stock[Ticker]	One to one	Single

Schema Diagram:



5. DATA PIPELINE

Pipeline Architecture:

```
Wikipedia (S&P 500 list)
         |
         v
Python: fetch_top25_sp500.py
         |
         |-- Step 1: Fetch S&P 500 company list
         |-- Step 2: Fetch market cap for all 503 companies
         |-- Step 3: Filter to Top 25 by market cap
         |-- Step 4: Download daily OHLCV price history
         |-- Step 5: Fetch company fundamentals
         |-- Step 6: Build dim_stock dimension table
         |-- Step 7: Save stock_prices.csv,
         |           fundamentals.csv, dim_stock.csv
         v
GitHub Repository (data/ folder)
         |
         v
Power BI Desktop
(reads raw GitHub URLs, press Alt + F5 to refresh)
```

GitHub Actions Workflow:

The pipeline is fully automated via a GitHub Actions workflow scheduled to run daily:

```yaml
name: Daily Stock Data Refresh
on:
  schedule:
    - cron: '0 6 * * *'    # 6:00 AM UTC daily
  workflow_dispatch:       # Manual trigger available
```

```
permissions:
  contents: write
```

Workflow steps:

1. Clone repository to virtual machine
2. Install Python 3.11 and required libraries
3. Execute fetch_top25_sp500.py
4. Commit updated CSVs back to repository

Libraries used: yfinance, pandas, requests, lxml

Data Sources:

Source	Data	Method	Frequency
Yahoo Finance (yfinance)	Daily OHLCV, market cap, PE ratio, avg volume	Python API library	Daily
Wikipedia	S&P 500 company list, sector, industry	requests and pd.read_html	Daily

6. DASHBOARD FEATURES

Overview:

The MarketPulse dashboard is built in Power BI Desktop with a dark financial terminal theme and consists of the following analytical components:

6.1 Stock Screener Table

A Yahoo Finance-style master table displaying all 25 tickers with the following metrics per row:

Column	Description
Ticker	Stock symbol
Company Name	Full company name
Price	Latest closing price
Previous Price	Most recent prior trading day close
Change	Absolute daily price movement
Change %	Percentage daily price movement (green/red)
Volume	Daily trading volume
Avg Volume (3M)	3-month average volume with data bars
Market Cap	Total market capitalization
PE Ratio (TTM)	Trailing twelve months PE ratio
52W High / Low	52-week price range
52W Change %	Year-over-year price performance

Interactive sorting via a tile slicer with three modes:

- Top Gainers: sorted by highest Change %
- Top Losers: sorted by lowest Change %
- Most Active: sorted by highest Volume

6.2 Price Trend with Moving Averages

A line chart showing three series for the selected ticker:

Series	Color	Purpose
Current Price	White	Primary price reference
30-Day MA	Blue (#4E79A7)	Short-term trend indicator
90-Day MA	Orange (#F28E2B)	Long-term trend indicator

Paired with a dynamic date range slicer: **3D, 7D, 1M, 6M, YTD, 1Y, All**

6.3 Volume Analysis Chart

A combo chart tracking:

- **Daily Volume:** column bars showing actual trading activity
- **Running Volume High:** a line showing cumulative peak volume, persisting forward until a new high is reached

This flags unusual trading activity where volume exceeds the historical peak, which is a standard indicator used in market analysis.

6.4 Risk vs Return Scatter Plot

Each of the 25 tickers plotted as a bubble with four dimensions:

Dimension	Measure
X-axis	Volatility (standard deviation of daily returns)
Y-axis	Total Return % since January 2023
Bubble size	Average 3-month trading volume
Bubble color	Sector classification

A vertical reference line at average volatility divides the chart into high-risk and low-risk zones, enabling instant portfolio-level comparison.

6.5 7-Day Price Change Waterfall Chart

A waterfall chart showing daily price change contribution for each of the last 7 trading days for the selected ticker. Green bars indicate positive days, and red bars indicate negative days, making it immediately clear which specific days drove the weekly movement.

6.6 Dashboard Controls

Control	Type	Function
Ticker selector	Slicer	Filters all charts to selected stock
Date range	Tile slicer	Controls time window for price and volume charts
Sort mode	Tile slicer	Controls screener table sort order
Reset button	Native button	Clears all selections back to default state

7. TECHNICAL IMPLEMENTATION

Key DAX Measures:

1. Previous Price - The most technically complex measure in the project. Standard Power BI time intelligence functions like DATEADD break on weekends and market holidays, returning blank values for non-trading days. The solution uses ALL() to remove filter context and explicit date filtering to always resolve the last available trading day:

```
Previous Price =
VAR CurrDate = MAX(stock_prices[Date])
VAR CurrTicker = SELECTEDVALUE(stock_prices[Ticker])
VAR PrevDate =
    CALCULATE(
        MAX(stock_prices[Date]),
        FILTER(
            ALL(stock_prices),
            stock_prices[Date] < CurrDate
        )
    )
RETURN
CALCULATE(
    MAX(stock_prices[Close]),
    stock_prices[Date] = PrevDate,
    stock_prices[Ticker] = CurrTicker
)
```

2. Running Volume High - Cumulative maximum volume that persists forward until a new peak is reached:

```
Running Volume High =
VAR CurrDate = MAX(stock_prices[Date])
VAR CurrTicker = SELECTEDVALUE(stock_prices[Ticker])
RETURN
CALCULATE(
    MAX(stock_prices[Volume]),
    FILTER(
        ALL(stock_prices),
        stock_prices[Ticker] = CurrTicker &&
        stock_prices[Date] <= CurrDate
    )
)
```

3. Dynamic Sort - Powers the Top Gainers / Top Losers / Most Active slicer by switching the sort key based on user selection. Named as a blank space to hide the helper column in the visual:

```
" " =
VAR SelectedOption =
    SELECTEDVALUE('Sort Selector'[Sort Type])
RETURN
SWITCH(
    SelectedOption,
    "Top Gainers",  [Change %],
    "Top Losers",  -[Change %],
    "Most Active",  [Volume (Measure)],
    [Change %]
)
```

Calculated Columns:

Column	Table	Logic
Previous Close (Col)	stock_prices	Row-level previous trading day close using TOPN, required for sparkline compatibility
Daily Change (Col)	stock_prices	Close minus Previous Close (Col) at row level
Daily Return (Col)	stock_prices	Daily Change (Col) divided by Previous Close (Col), used for volatility calculation

Dashboard Theme:

Applied via a custom JSON theme file controlling all visual states globally, including canvas background, visual backgrounds, text colors, axis labels, table alternating rows, and slicer selected/unselected states. This bypasses Power BI Free tier UI limitations for slicer formatting.

Element	Color
Canvas background	#0D1117
Visual background	#161B22
Positive values	#00B050
Negative values	#FF4444
Axis labels	#8B949E

8. CHALLENGES AND SOLUTIONS

8.1 DAX Time Intelligence: Weekend and Holiday Gaps

Challenge: Standard DATEADD function assumes continuous calendar dates. Stock markets only trade on weekdays excluding public holidays. Using DATEADD(-1, DAY) returned blank for every Monday and the day after every holiday.

Attempted approaches: DATEADD, date minus one arithmetic, TOPN-based filtering

Solution: Custom Previous Price measure using ALL() to remove filter context and FILTER to explicitly find the maximum date prior to the current date, resolving the last real trading day regardless of calendar gaps.

Learning: Power BI time intelligence functions are designed for continuous date series. Financial data requires custom logic that respects market calendar gaps.

8.2 Sparklines: Row Context vs Filter Context

Challenge: Power BI sparklines require a physical time series value per row. Dynamic DAX measures evaluate in filter context which is incompatible with the sparkline engine. Multiple approaches were attempted including SELECTEDVALUE wrappers and measure restructuring, all of which failed silently.

Attempted approaches: Dynamic measures, SELECTEDVALUE wrappers, Python visual with matplotlib

Solution: Moved daily change calculation to a calculated column (Daily Change Col) to ensure row-level physical values. This resolved sparkline rendering but required understanding the fundamental difference between row context (columns) and filter context (measures) in DAX.

Judgment call: After exhausting sparkline options and finding that Python charts were not dynamically responsive to slicer selections, the decision was made to replace the sparkline entirely with a more informative price analysis visual. Recognizing when to pivot rather than forcing a failing solution is a more valuable outcome than a working sparkline.

8.3 Wikipedia HTTP 403 Block

Challenge: Wikipedia blocked automated data requests from the pd.read_html() function, returning HTTP Error 403 Forbidden.

Solution: Added browser-style User-Agent headers to the requests.get() call, mimicking a real browser visit. One additional line of code resolved the issue entirely.

8.4 GitHub Actions: Permission Denial on Push

Challenge: The initial GitHub Actions workflow ran successfully but failed at the git push step with error 403 Permission Denied. GitHub tightened default GITHUB_TOKEN permissions from read-write to read-only between the manual test run and the first scheduled run.

Solution: Added explicit permissions: contents: write to the workflow file and passed the GITHUB_TOKEN to the checkout action. Additionally implemented conditional git diff logic to exit cleanly on non-trading days when no new data exists to commit.

8.5 Power BI Free Tier Limitations

Challenge: Power BI Free does not support scheduled cloud refresh, live dashboard sharing, or full slicer state formatting through the UI.

Solutions:

- Scheduled refresh was replaced by GitHub Actions automated CSV update. Power BI reads latest data from GitHub raw URLs on manual refresh.
- Dashboard sharing is handled through the GitHub repository and a downloadable .pbix file.
- Slicer formatting was applied via a custom JSON theme file, bypassing UI limitations entirely.

8.6 Many-to-Many Relationship Error

Challenge: Initial data model connected fundamentals directly to stock_prices, creating a many-to-many relationship that Power BI could not resolve cleanly.

Solution: Restructured to a snowflake schema routing fundamentals through dim_stock. Fundamentals connects to Dim_Stock which connects to stock_prices. This eliminated the ambiguous filter path and correctly separated financial metadata from price data.

9. RESULTS AND IMPACT

What Was Delivered:

Component	Status	Notes
Automated Python ETL pipeline	Complete	Runs daily via GitHub Actions
Power BI dashboard with 6 views	Complete	Dark theme, fully interactive
Snowflake schema data model	Complete	4 core tables, 2 disconnected
GitHub Actions automation	Complete	Daily 6AM UTC, manual trigger available
Power BI to GitHub connection	Complete	Raw URL data source, no local files needed
Technical documentation	Complete	Data dictionary, runbook, SOP
GitHub README	Complete	Architecture diagrams, DAX samples, schema
LinkedIn post	Complete	Scheduled for publication

Skills Demonstrated:

Skill Category	Specific Demonstration
Data Engineering	API integration, automated ETL, pipeline scheduling
Data Modeling	Snowflake schema, relationship management, calculated columns
DAX Development	Time intelligence, running calculations, dynamic sorting, filter context
Dashboard Design	Dark theme, conditional formatting, interactive controls
Python	pandas, yfinance, requests, web scraping, data transformation
Automation	GitHub Actions, cron scheduling, git operations
Documentation	Data dictionary, runbook, process mapping, reporting standards
Financial Domain	Moving averages, volatility, PE ratio, market cap, 52W metrics

Problem Solving	6 documented technical challenges each with attempted and final solutions

Portfolio Value:

This project demonstrates an end-to-end data analytics workflow that combines skills typically shown in isolation across separate portfolio projects. The deliberate constraint of using only free tools makes it directly relevant to analyst roles in organizations where paid tooling is not always available.

10. LIMITATIONS AND FUTURE ENHANCEMENTS

Current Limitations:

Limitation	Impact	Root Cause
Manual Power BI refresh required	Dashboard does not auto-update	Power BI Free tier, no scheduled cloud refresh
End-of-day data only	No intraday or real-time prices	yfinance limitation
US markets only	No international stocks	S&P 500 scope decision
No live sharing without Pro license	Requires .pbix download to view	Power BI Free tier
yfinance data gaps	Occasional missing values	Yahoo Finance API instability
Static top 25 at runtime	List does not rebalance intraday	Design decision for stability

Future Enhancements:

Enhancement	Value	Effort
Full S&P 500 coverage	More comprehensive market view	Medium
Intraday data via Alpha Vantage or Polygon.io	Live price updates during market hours	High
Portfolio simulator	Custom watchlist with simulated P&L tracking	Medium
Earnings calendar integration	Anticipate volatility windows per ticker	Low
News sentiment scoring	Contextual signal per ticker	High
Power BI mobile layout	Second page optimised for mobile viewing	Low

Daily email digest	Automated top movers summary via Gmail SMTP	Medium
Streamlit web version	Publicly shareable URL without Power BI Desktop	Medium

11. CONCLUSION

MarketPulse demonstrates that a production-quality, end-to-end data analytics solution can be built entirely within free tooling when the underlying data engineering, modeling, and design decisions are sound.

The project was built iteratively rather than from a predefined plan. Each technical challenge encountered, from DAX filter context issues to GitHub Actions permission errors, produced a more robust solution than the initial approach would have delivered. The willingness to abandon approaches that did not work, most notably the sparkline implementation, and replace them with cleaner alternatives reflects the kind of practical judgment that distinguishes functional analytics from theoretical knowledge.

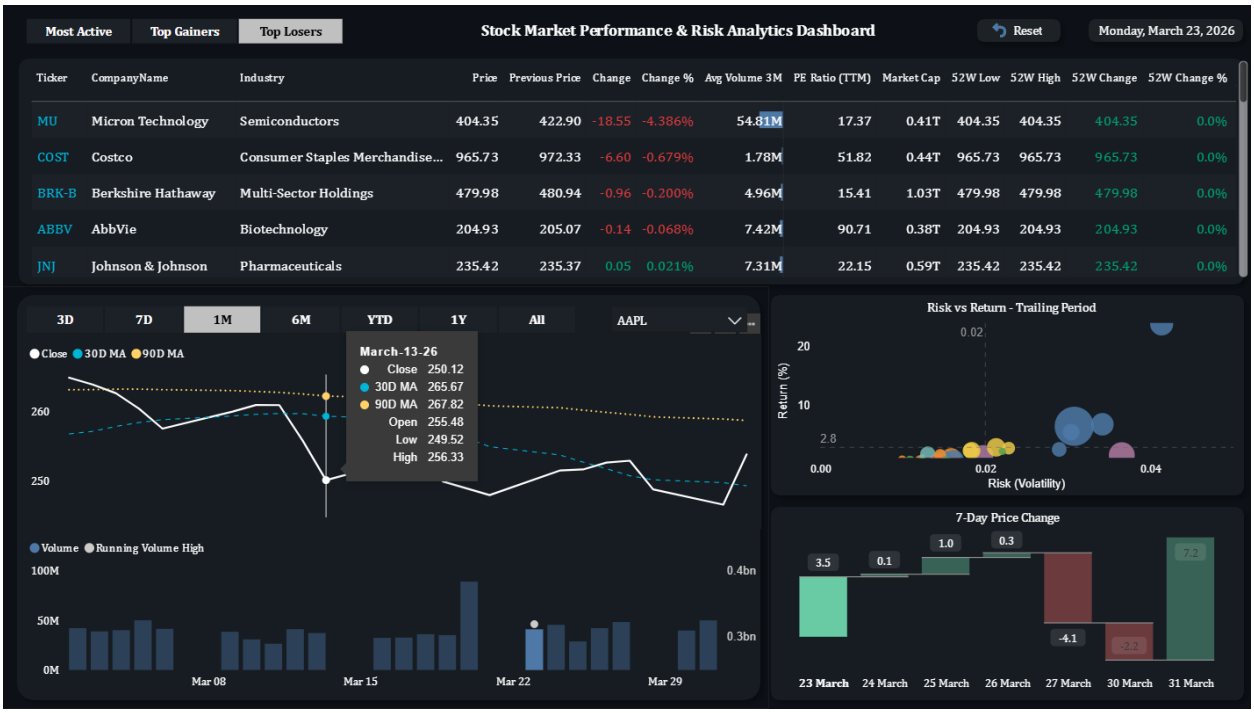
The documentation produced alongside the dashboard, including the data dictionary, runbook, SOP, process mapping and reporting standards, reflects an understanding that analytics work is only complete when it can be handed over and maintained by someone who was not involved in building it.

For a recruiter or hiring manager, this project represents evidence of the following:

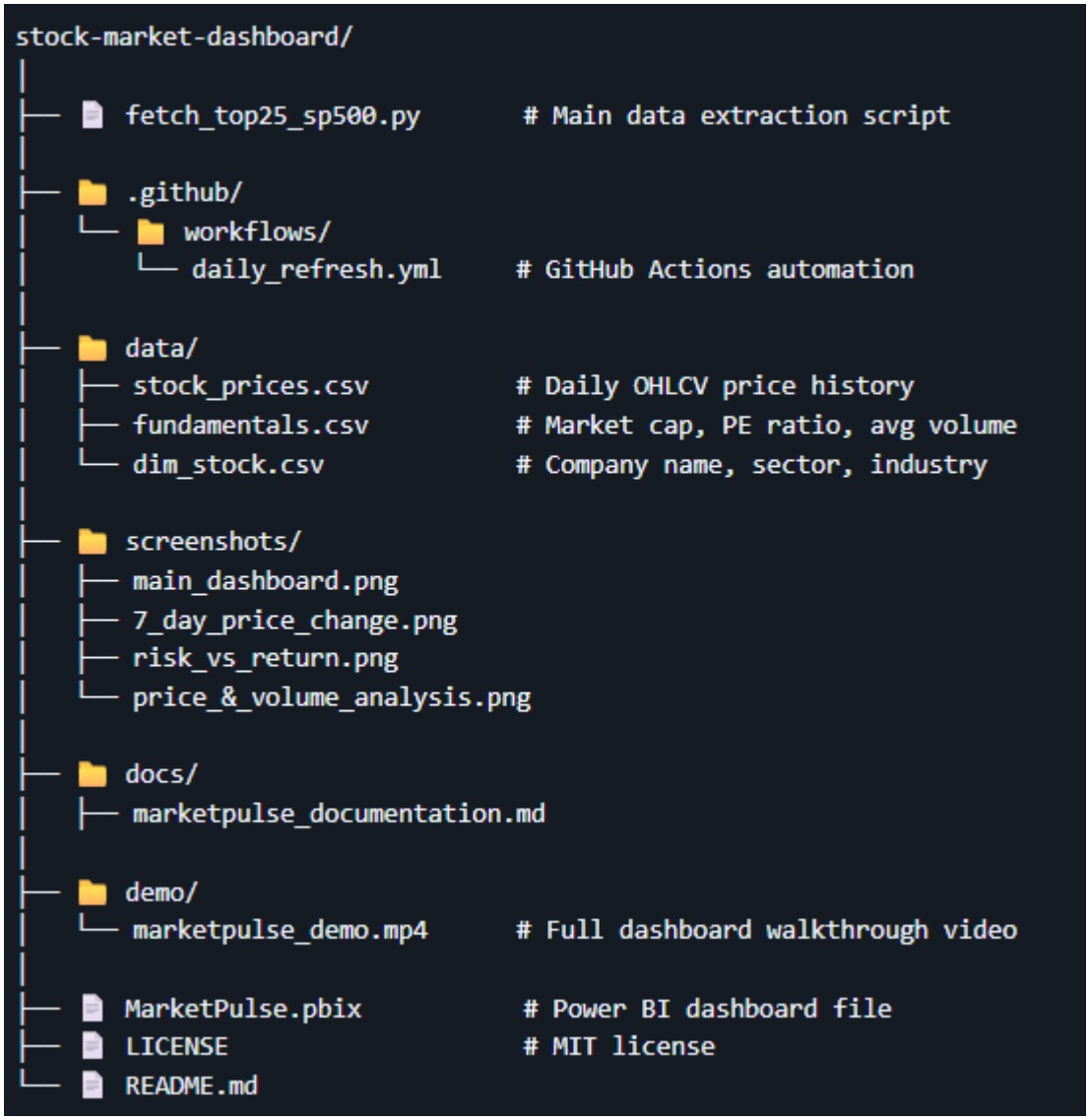
- The ability to independently scope, build and deliver an analytics project from raw data to documented output
- Proficiency across the full data analyst stack, covering Python, SQL-equivalent DAX logic, BI tooling, and pipeline automation
- The judgment to work within real constraints rather than ideal conditions
- The communication habit of documenting decisions, not just outputs

12. APPENDICES

Appendix A: MarketPulse Dashboard



Appendix B: Repository Structure



Appendix C: Tech Stack

Tool	Version	Purpose
Python	3.11	Data extraction and transformation
yfinance	Latest	Yahoo Finance API wrapper
pandas	Latest	Data manipulation and CSV output
requests	Latest	HTTP requests for Wikipedia scraping
lxml	Latest	HTML parsing for pd.read_html()
Power BI Desktop	Free	Dashboard design and DAX modeling
GitHub Actions	N/A	Pipeline scheduling and automation
GitHub	N/A	Version control and data storage

Appendix D: DAX Measures Reference

Measure	Purpose
Previous Price	Last available trading day close per ticker
Change	Absolute daily price movement
Change %	Percentage daily price movement
30D MA	30-day moving average
90D MA	90-day moving average
Running Volume High	Cumulative peak volume
Volatility	Standard deviation of daily returns
Total Return %	Performance from first to latest date
Avg Volatility	Average volatility across all tickers
" " Dynamic Sort	Table sort key driven by slicer selection
Date Filter	Binary flag for date range filtering
Range Start Date	Calculated start date from selected range

Appendix E: Naming Conventions

Element	Convention	Example
DAX Measures	Title case with spaces	Previous Price, Running Volume High
Calculated Columns	Title case with (Col) suffix	Daily Change (Col), Daily Return (Col)
Disconnected Tables	Title case descriptive	Date Range, Sort Selector
CSV files	Lowercase underscores	stock_prices.csv
Python variables	Lowercase underscores	stock_prices_df
Python constants	Uppercase underscores	TICKERS, START_DATE

Appendix F: Color and Formatting Standards

Element	Color	Hex
Canvas background	Deep navy	#0D1117
Visual background	Dark slate	#161B22
Card background	Slightly lighter	#1C2128
Positive / Gain	Green	#00B050
Negative / Loss	Red	#FF4444
Price line	White	#FFFFFF
30D MA	Blue	#4E79A7
90D MA	Orange	#F28E2B
Axis labels	Grey	#8B949E